

Software Timer

Software Timer

1. Software Timer Overview
2. Core concepts
 - 2.1 How Software Timers Work
 - 2.2 Timer Creation and Start API
 - 2.3 Callback Function Signature
 - 2.4 Periodic Timer vs One-Shot Timer
3. Project Architecture and Flow
4. Hardware Dependencies
5. Source Code Analysis
 - 5.1 Header Files and Macro Configuration
 - 5.2 key_scan() — Button Debounce Scan
 - 5.3 brightness_timer_callback() — Timer Callback
 - 5.4 app_main() — Main Entry
6. Operation Flow
 - 6.1 Compilation and Flashing Flow
 - 6.2 Normal Operation
7. FAQ

1. Software Timer Overview

Software Timer is a pure software timing function provided by FreeRTOS, uniformly scheduled by the kernel's **Timer Service Task**. When the timer expires, its **callback function executes in the context of the Timer Service Task** (not interrupt context), so most FreeRTOS APIs can be called in the callback.

Importance and Application Scenarios: Software timers are one of the most commonly used features in embedded real-time operating systems. Compared to hardware timers, software timers don't occupy hardware resources and their quantity isn't limited by chip peripherals. Additionally, callbacks execute in task context, making programming safer. Typical application scenarios include:

- **Periodic sensor polling:** Periodically collect temperature, humidity, light and other sensor data without occupying a task.
- **LED effects and breathing light:** Periodically adjust PWM duty cycle or color values to achieve smooth gradient effects.
- **Timeout protection and watchdog feeding:** Set timeout detection for communication protocols, user operations, or periodically feed hardware watchdog.
- **Heartbeat signals and status reporting:** Periodically send device heartbeat packets or status info to host/cloud platform.
- **Button debouncing and long-press detection:** Use one-shot timers to implement button debounce delay and long-press judgment.

This project demonstrates: 50ms period software timer drives WS2812 to implement **blue breathing light (periodic mode)** and **red breathing light (one-shot mode)**. Button switches between the two modes. Bright/dim ratio is 2:1.

2. Core concepts

2.1 How Software Timers Work

```
FreeRTOS Kernel
|
├─ IDLE Task
├─ User Task
└─ Timer Service Task (Daemon)
    |
    ├─ Maintain timer linked list
    ├─ Check timeout
    ├─ Call callback function
    └─ Reload (auto-reload=true)
```

2.2 Timer Creation and Start API

```
// Create timer
TimerHandle_t xTimerCreate(
    const char *pcTimerName,      // Timer name (debug)
    TickType_t xTimerPeriod,      // Period (ticks)
    UBaseType_t uxAutoReload,
    void *pvTimerID,              // Timer ID
    TimerCallbackFunction_t pxCallbackFunction // callback
);

// Start timer
BaseType_t xTimerStart(
    TimerHandle_t xTimer,
    TickType_t xTicksToWait       // Timeout (usually 0)
);

// Stop timer
BaseType_t xTimerStop(
    TimerHandle_t xTimer,
    TickType_t xTicksToWait
);
```

2.3 Callback Function Signature

```
void vTimerCallback(TimerHandle_t xTimer)
{
    // Execute in Timer Service Task context
    // Can call FreeRTOS API
    // But should NOT call vTaskDelay (will block timer service)
}
```

2.4 Periodic Timer vs One-Shot Timer

The following table compares the two timer types from multiple dimensions:

Comparison Dimension	Periodic Timer (pdTRUE)	One-Shot Timer (pdFALSE)
Creation parameter	<code>xTimerCreate(..., pdTRUE, ...)</code>	<code>xTimerCreate(..., pdFALSE, ...)</code>
Trigger behavior	Auto-reload after expiration, infinite looping	Auto-stop after expiration, trigger only once
Restart method	Auto-restart on expiration, no manual intervention	Need to call <code>xTimerStart()</code> again to restart
Stop method	Actively call <code>xTimerStop()</code>	Auto-stop on expiration, can also call <code>xTimerStop()</code>
Callback context	Timer service task context	Timer service task context (same as periodic)
Callback duration	Should be as short as possible to avoid blocking subsequent triggers	Should also be short, but one-shot has slightly higher fault tolerance
API complexity	Simple: just start after creation, no lifecycle management	Medium: need to call <code>xTimerStart()</code> at appropriate time to restart
Applicable scenarios	Sensor polling, LED blinking/breathing, heartbeat, PWM regulation, display refresh	Timeout protection, watchdog feeding, delayed operations, button debounce, communication response waiting
Resource usage	Continuously occupies one node in timer list	Removed from timer list after expiration, releases resources
Example	50ms period, trigger every 50ms → never stops	3s timeout, trigger once after 3s → auto-stop
Change period	<code>xTimerChangePeriod()</code> takes effect immediately and resets current timer	<code>xTimerChangePeriod()</code> only takes effect after timer is restarted

Practical Tip: In this project, both timer behaviors are implemented by simulating on **the same periodic timer** via the `is_periodic` flag, rather than truly creating two timers. Advantages of this approach:

- 1. Occupies single timer resource, no timer creation/destruction overhead during mode switching.
 - 2. Mode switching only flips the flag, no creation/destruction overhead.
 - 3. All LED operations are unified in the same callback, avoiding race conditions between multiple timers.
- In this project, the `paused` variable essentially achieves the effect of "one-shot timer stops after completion".

3. Project Architecture and Flow

```
app_main()
├─ rgb_init()           ← WS2812 (GPIO48)
├─ key_init()           ← GPIO0 button (polling, no interrupt)
├─ xTimerCreate(50ms, pdTRUE) ← 50ms timer (auto-reload, never stops)
└─ xTimerStart()

Main loop (20ms polling)
└─ key_scan() → mode_switch_requested = true

Timer callback (50ms) – Timer service task context
├─ Handle switch request → is_periodic = !is_periodic
├─ paused? → direct return
├─ Periodic mode (is_periodic=true):
│   Blue breathing, bright/dim ratio 2:1
│   Fade in: 0→255, step 5, 51 steps×50ms≈2.55s
│   Fade out: 255→0, step 10, 26 steps×50ms≈1.30s
│   Continue next cycle after period ends
│
└─ One-shot mode (is_periodic=false):
    Red breathing, bright/dim ratio 2:1
    One cycle complete → paused=true → LED off
```

Breathing Light Parameters:

Parameter	Periodic Mode (Blue)	One-Shot Mode (Red)
Timer period	50ms	50ms
Fade-in step	+5	+5
Fade-out step	-10	-10
Fade-in time	~2.55s	~2.55s
Fade-out time	~1.30s	~1.30s
Complete period	~3.85s	~3.85s (then pause)
Bright/dim ratio	2 : 1	2 : 1

4. Hardware Dependencies

Pin	Function	Description
GPIO 0	BOOT button	Polling mode, press once to switch mode
GPIO 48	WS2812 RGB LED	RMT peripheral driver

Required Components: `freertos/FreeRTOS`, `freertos/task`, `freertos/timers`, `driver/gpio`, `led_strip`, `esp_log`

This project uses a **third-party library** again: For details, review (**Third-party library installation process: 2.Basic Peripherals → 10. RGB Colorful Light Experiment**)

Note: `led_strip` is a third-party component provided by ESP-IDF Component Registry, not built into IDF. Before use, declare dependency in `main/idf_component.yml`:

```
## IDF Component Manager Manifest File
dependencies:
idf:
  version: '>=4.1.0'
  espressif/led_strip: ^3.0.3
```

After declaration, execute `idf.py build`, and component manager will automatically download component to `managed_components/` directory.

5. Source Code Analysis

Full source code: `Source Code` → `ESP-IDF` → `4.FreeRTOS` → `3. RTOS_Timer`

5.1 Header Files and Macro Configuration

The following code introduces headers for FreeRTOS timer, GPIO driver, WS2812 strip driver, and defines macros for timer period, brightness step, button debounce and other key parameters, along with global state variable declarations.

```
#include "freertos/FreeRTOS.h"           // FreeRTOS core
#include "freertos/task.h"               // Task API
#include "freertos/timers.h"            // Software timer API
#include "driver/gpio.h"                 // GPIO driver
#include "led_strip.h"
#include "esp_log.h"                     // Log output

#define RGB_GPIO          48             // RGB LED data pin
#define KEY_GPIO          0              // Button GPIO pin
#define LED_NUM            1             // Number of LEDs
#define TIMER_PERIOD_MS   50            // Timer period in ms
#define BRIGHTNESS_STEP_UP 5            // Fade-in step (small)
#define BRIGHTNESS_STEP_DN 10           // Fade-out step (large)
#define BRIGHTNESS_MIN    0             // Minimum brightness
```

```

#define BRIGHTNESS_MAX      255      // Maximum brightness
#define KEY_DEBOUNCE_MS     50

static const char *TAG = "TIMER_DEMO";
static led_strip_handle_t led_strip;
static TimerHandle_t brightness_timer;
static int brightness = 0;
static bool fade_up = true;
static bool is_periodic = true;
static bool paused = false;
static TickType_t cycle_start_ticks = 0;
static volatile bool mode_switch_requested = false;

```

5.2 key_scan() — Button Debounce Scan

This function implements button debouncing via software delay, waits for button release after detecting valid press before returning, avoiding one press triggering multiple actions.

```

bool key_scan(void)
{
    if (gpio_get_level(KEY_GPIO) == 0)
    {
        vTaskDelay(KEY_DEBOUNCE_MS / portTICK_PERIOD_MS);
        if (gpio_get_level(KEY_GPIO) == 0)
        {
            while (gpio_get_level(KEY_GPIO) == 0)
            {
                vTaskDelay(10 / portTICK_PERIOD_MS);
            }
            return true;
        }
    }
    return false;
}

```

5.3 brightness_timer_callback() — Timer Callback

Called once every 50ms by the timer service task, executes all brightness gradient logic according to current mode. All LED operations are completed in the callback to avoid race conditions with the main loop.

```

void brightness_timer_callback(TimerHandle_t timer)
{
    if (mode_switch_requested)
    {
        mode_switch_requested = false;
        is_periodic = !is_periodic;
        brightness = BRIGHTNESS_MIN;
        fade_up = true;
        paused = false;
        led_strip_clear(led_strip);
    }
}

```

```

    if (is_periodic)
        ESP_LOGI(TAG, "Switched to: Periodic Timer (Blue, cycling)");
    else
        ESP_LOGI(TAG, "Switched to: One-Shot Timer (Red, single cycle)");
}

if (paused)
    return;

if (fade_up)
{
    if (brightness == BRIGHTNESS_MIN)
        cycle_start_ticks = xTaskGetTickCount();

    brightness += BRIGHTNESS_STEP_UP;
    if (brightness >= BRIGHTNESS_MAX)
    {
        brightness = BRIGHTNESS_MAX;
        fade_up = false;
    }
}
else
{
    brightness -= BRIGHTNESS_STEP_DN;
    if (brightness <= BRIGHTNESS_MIN)
    {
        brightness = BRIGHTNESS_MIN;
        fade_up = true;
    }

    TickType_t elapsed = xTaskGetTickCount() - cycle_start_ticks;
    uint32_t elapsed_ms = elapsed * portTICK_PERIOD_MS;
    ESP_LOGI(TAG, "Cycle completed: %lu ms (fade-in ~2.55s, fade-out ~1.30s)",
elapsed_ms);

    if (!is_periodic)
    {
        led_strip_clear(led_strip);
        paused = true;
        ESP_LOGI(TAG, "One-shot finished");
        return;
    }
}

if (is_periodic)
    led_strip_set_pixel(led_strip, 0, 0, 0, brightness);
else
    led_strip_set_pixel(led_strip, 0, brightness, 0, 0);

led_strip_refresh(led_strip);
}

```

Why does main loop only set flags? To avoid race conditions for variables like `is_periodic`, `brightness` between main loop and timer callback. All LED operations and state changes are unified in the callback. Timer never stops (`xTimerCreate` specifies `pdTRUE` auto-reload), pause/switch is implemented only via `paused` / `mode_switch_requested` flags.

5.4 app_main() — Main Entry

Main function sequentially completes WS2812 strip initialization, button GPIO configuration, software timer creation and start, then enters main loop to poll button and set switch request flag.

```
void app_main(void)
{
    led_strip_config_t strip_config = {
        .strip_gpio_num = RGB_GPIO,
        .max_leds = LED_NUM,
        .led_model = LED_MODEL_WS2812,
        .color_component_format = LED_STRIP_COLOR_COMPONENT_FMT_GRB,
        .flags.invert_out = false,
    };

    led_strip_rmt_config_t rmt_config = {
        .resolution_hz = 10 * 1000 * 1000,
        .flags.with_dma = false,
    };

    ESP_ERROR_CHECK(led_strip_new_rmt_device(&strip_config, &rmt_config, &led_strip));
    led_strip_clear(led_strip);
    gpio_config_t key_config = {
        .pin_bit_mask = 1ULL << KEY_GPIO,
        .mode = GPIO_MODE_INPUT,
        .pull_up_en = GPIO_PULLUP_ENABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE,
    };

    gpio_config(&key_config);

    brightness_timer = xTimerCreate(
        "Brightness",
        pdMS_TO_TICKS(TIMER_PERIOD_MS),
        pdTRUE,
        0,
        brightness_timer_callback
    );

    xTimerStart(brightness_timer, 0);

    ESP_LOGI(TAG, "Periodic timer started: Blue LED cycling, press KEY(GPIO0) to switch");

    while (1)
    {
        if (key_scan())
        {
            mode_switch_requested = true;
        }
        vTaskDelay(20 / portTICK_PERIOD_MS);
    }
}
```



```
}  
}
```

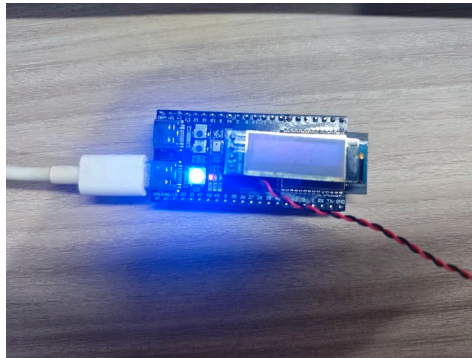
6. Operation Flow

6.1 Compilation and Flashing Flow

For detailed flashing process: 1. Before Development → 3. Build and Flash

6.2 Normal Operation

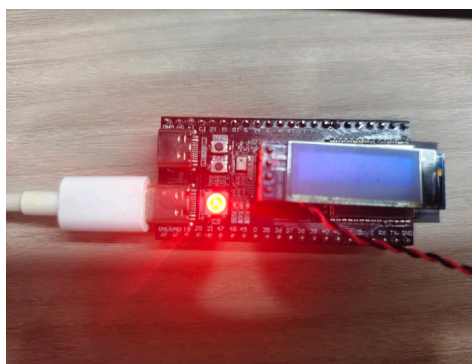
1. After flashing, power on → **Default periodic mode** → Blue breathing light continuously cycles



2. Serial prints elapsed time at end of each cycle "Cycle completed: ~3850 ms"

```
I (29832) TIMER_DEMO: Cycle completed: 3800 ms (fade-in ~2.55s, fade-out ~1.30s)  
I (33682) TIMER_DEMO: Cycle completed: 3800 ms (fade-in ~2.55s, fade-out ~1.30s)  
I (37532) TIMER_DEMO: Cycle completed: 3800 ms (fade-in ~2.55s, fade-out ~1.30s)  
I (41382) TIMER_DEMO: Cycle completed: 3800 ms (fade-in ~2.55s, fade-out ~1.30s)  
I (45232) TIMER_DEMO: Cycle completed: 3800 ms (fade-in ~2.55s, fade-out ~1.30s)  
I (49082) TIMER_DEMO: Cycle completed: 3800 ms (fade-in ~2.55s, fade-out ~1.30s)
```

3. Press **BOOT** button once → Switch to **one-shot mode** → Red breathing light turns off after one cycle



```
I (6632) TIMER_DEMO: Switched to: One-Shot Timer (Red, single cycle)  
I (10432) TIMER_DEMO: Cycle completed: 3800 ms (fade-in ~2.55s, fade-out ~1.30s)  
I (10432) TIMER_DEMO: One-shot finished
```

4. Press **again** → Switch back to periodic mode → Blue breathing light resumes

```
I (10632) TIMER_DEMO: Switched to: Periodic Timer (Blue, cycling)  
I (14432) TIMER_DEMO: Cycle completed: 3800 ms (fade-in ~2.55s, fade-out ~1.30s)
```

Log example:

```
I (282) TIMER_DEMO: Periodic timer started: Blue LED cycling, press KEY(GPIO0) to switch
I (4132) TIMER_DEMO: Cycle completed: 3800 ms (fade-in ~2.55s, fade-out ~1.30s)
I (5882) TIMER_DEMO: Switched to: One-Shot Timer (Red, single cycle)
I (9682) TIMER_DEMO: Cycle completed: 3800 ms (fade-in ~2.55s, fade-out ~1.30s)
I (9682) TIMER_DEMO: One-shot finished
I (10632) TIMER_DEMO: Switched to: Periodic Timer (Blue, cycling)
I (14432) TIMER_DEMO: Cycle completed: 3800 ms (fade-in ~2.55s, fade-out ~1.30s)
```

```
I (282) TIMER_DEMO: Periodic timer started: Blue LED cycling, press KEY(GPIO0) to switch
I (4132) TIMER_DEMO: Cycle completed: 3800 ms (fade-in ~2.55s, fade-out ~1.30s)
I (5882) TIMER_DEMO: Switched to: One-Shot Timer (Red, single cycle)
I (9682) TIMER_DEMO: Cycle completed: 3800 ms (fade-in ~2.55s, fade-out ~1.30s)
I (9682) TIMER_DEMO: One-shot finished
I (10632) TIMER_DEMO: Switched to: Periodic Timer (Blue, cycling)
I (14432) TIMER_DEMO: Cycle completed: 3800 ms (fade-in ~2.55s, fade-out ~1.30s)
I (18282) TIMER_DEMO: Cycle completed: 3800 ms (fade-in ~2.55s, fade-out ~1.30s)
```

7. FAQ

Symptom	Cause	Solution
Timer not triggering	<code>xTimerStart</code> not called / Timer service task not started	Confirm start call; check <code>configUSE_TIMERS=1</code>
Breathing light jitter	Timer period too large	Reduce period (recommended $\leq 50\text{ms}$); increase step
Button switch ineffective	Debounce time too short	Increase <code>KEY_DEBOUNCE_MS</code>
One-shot mode doesn't pause	<code>is_periodic</code> judgment missed	Check <code>is_periodic</code> at brightness zero and set <code>paused=true</code>
LED flickers during mode switch	Switch and LED update race	Unified handling in callback: switch flag + LED operation
Calling <code>vTaskDelay</code> in callback	Cannot — will block timer service	Callback should return quickly; use state machine instead of delay